src/dynamics.py · state_derivative MMA3001 DeepONet project

state_derivative

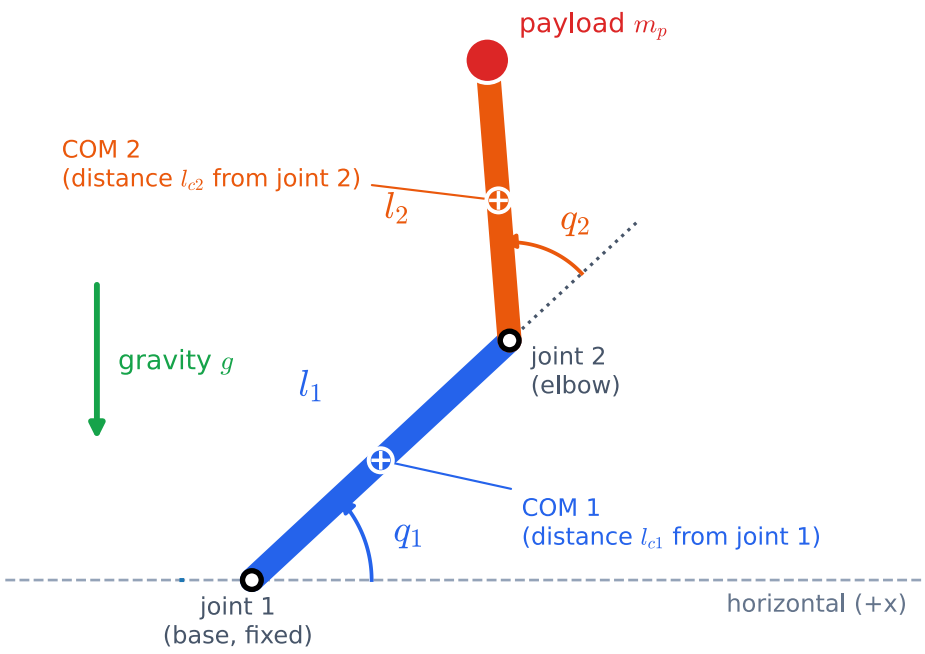
The right-hand side of the simulation: from M, Coriolis, gravity, friction and motor torque to the state's rate of change

In this document

1. The question
2. Toolbox: ODEs and the state
3. Assembling the equation of motion
4. Solving for accelerations
5. Worked examples
6. Code-maths dictionary
7. Things worth knowing

The set-up: a two-link robot arm in a vertical plane

Everything in this project describes one physical object: a planar robot arm with two rigid links, two motorised joints, bolted to a wall at the origin. It moves in a *vertical* plane, so gravity matters. At the tip of link 2 we may clip on an extra weight (the **payload**). Each joint has a little friction that slows it down in proportion to how fast it spins.



The arm. Blue = link 1, orange = link 2. The angle q_1 is measured from the horizontal; the angle q_2 is measured from the *straight-on extension* of link 1 (dotted), not from the horizontal. The dots with a white cross are the links' centres of mass (COM).

Conventions (fixed everywhere in the project): the state is $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]$; gravity points in the $-y$ direction; potential energy is zero at the height of joint 1; positive angles and torques are anticlockwise.

Symbol	Meaning	Value in the code	Unit
$l_1,\ l_2$	link lengths (joint to joint)	0.5, 0.4	m

$m_1,\ m_2$	link masses (without payload)	2.0, 1.5	kg
$l_{c1},\ l_{c2}$	distance from a link's joint to its own centre of mass (COM)	0.25, 0.2 (middle of each link)	m
$I_1,\ I_2$	moment of inertia of a link about its own COM	$ml^2/12$ (uniform rod)	kg m^2
g	gravitational acceleration	9.81	m/s^2
m_p	point-mass payload at the tip of link 2 (varies per sample)	0 to 1 (dataset), up to 1.5 out-of-range	kg
b	viscous joint friction coefficient (varies per sample)	0.05 to 0.5	N m s/rad
$q_1,\ q_2$	joint angles: q_1 from the horizontal, q_2 relative to link 1	state	rad
$\dot{q}_1,\ \dot{q}_2$	joint angular velocities (a dot means d/dt)	state	rad/s
$\tau_1,\ \tau_2$	motor torques applied at joints 1 and 2 (the inputs)	$A\sin(2\pi f t+\phi)$	N m

Toolbox: Lagrangian mechanics in five minutes

You know Newton's second law for a straight-line motion: $F = ma$. For something that *rotates* about a fixed pin the same law reads

$$\tau = I\,\ddot{\theta},$$

where τ is torque (a twisting push, N m), I the moment of inertia (resistance to being spun, kg m^2) and $\ddot{\theta}$ the angular acceleration. For *one* link that is easy. For *two* links, the second link is moved by the first, which is moved by the base, and the joints push on each other with forces we do not know. Newton's method forces us to compute all those internal forces. **Lagrange's trick** avoids them completely by working with *energies*, which are plain numbers.

The recipe

1. Choose coordinates that describe the pose with no redundancy: here q_1, q_2 (the *generalised coordinates*).
2. Compute the kinetic energy T (as a function of $q, \dot{q}$) and the potential energy V (as a function of q).
3. For every coordinate q_k evaluate the **Euler-Lagrange equation**

$$\underbrace{\frac{d}{dt}\left(\frac{\partial T}{\partial \dot{q}_k}\right)}_{\text{(a) rate of change of 'momentum'}} - \underbrace{\frac{\partial T}{\partial q_k}}_{\text{(b) pose-dependence of } T} + \underbrace{\frac{\partial V}{\partial q_k}}_{\text{(c) gravity pulls}} = Q_k$$

   where Q_k collects everything we push with from outside (motor torque τ_k, friction).

Reading it: (a) is "how fast is the generalised momentum $\partial T/\partial \dot{q}_k$ changing" (it is the analogue of $d(mv)/dt$); (c) is the force that gravity exerts along coordinate q_k, namely minus the slope of the potential energy (a ball rolls downhill, i.e. the force is $-\partial V/\partial q$, so moving it to the other side of the equation gives $+\partial V/\partial q$); (b) is a correction that appears because the "mass" itself can depend on the pose.

The symbol ∂ means a **partial derivative**: differentiate with respect to one variable while pretending all the others are constants. For instance if $T = \tfrac{1}{2}(3+\cos q_2)\,\dot{q}_1^2$ then $\partial T/\partial \dot{q}_1 = (3+\cos q_2)\dot{q}_1$ and $\partial T/\partial q_2 = -\tfrac{1}{2}\sin q_2\,\dot{q}_1^2$.

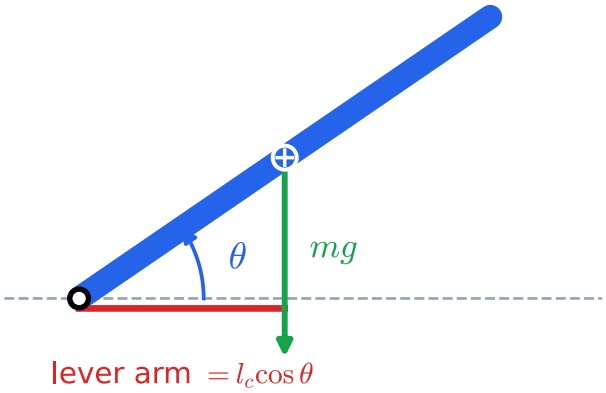
Test run: one swinging rod (you can check it with Newton)

A uniform rod of mass m pivots at the origin, angle θ from the horizontal; its COM is a distance l_c from the pivot. Its inertia about the pivot is $I_p = I_{\text{cm}} + ml_c^2$ (explained in the *combine_payload* document). Then

$$T = \tfrac{1}{2} I_p \dot{\theta}^2, \qquad V = mg\, l_c \sin\theta.$$

Euler-Lagrange: $\frac{d}{dt}(I_p\dot{\theta}) - 0 + mgl_c\cos\theta = \tau$, i.e.

$$I_p\ddot{\theta} + mg\, l_c \cos\theta = \tau.$$



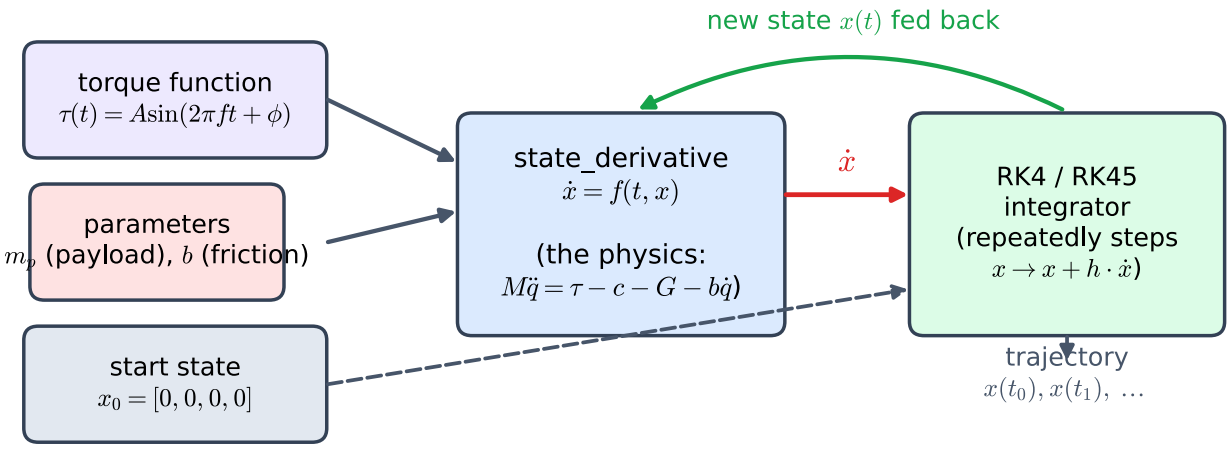
Newton's view of the same rod. Gravity pulls down with force mg at the COM. Only the horizontal distance $l_c\cos\theta$ to the pivot gives a twist (the lever arm), so gravity's torque about the pin is $mg\, l_c\cos\theta$ - exactly the term Lagrange produced.

Newton gives $I_p\ddot{\theta} = \tau - (\text{gravity torque})$, the same equation. So the method works, and for two links it needs only energies. That is the whole idea; the rest is careful algebra.

1 The question this function answers

All the earlier pieces (M, c, G) describe the arm *at one instant*. To **simulate** the arm we need to predict how it moves over time. The universal recipe for that (used by every ODE solver, from RK4 to MATLAB's `ode45`) is: "*tell me the current state and I will tell you how fast each state variable is changing right now; I will take a tiny step in that direction and ask again.*" `state_derivative` is the thing that answers "how fast is everything changing right now?":

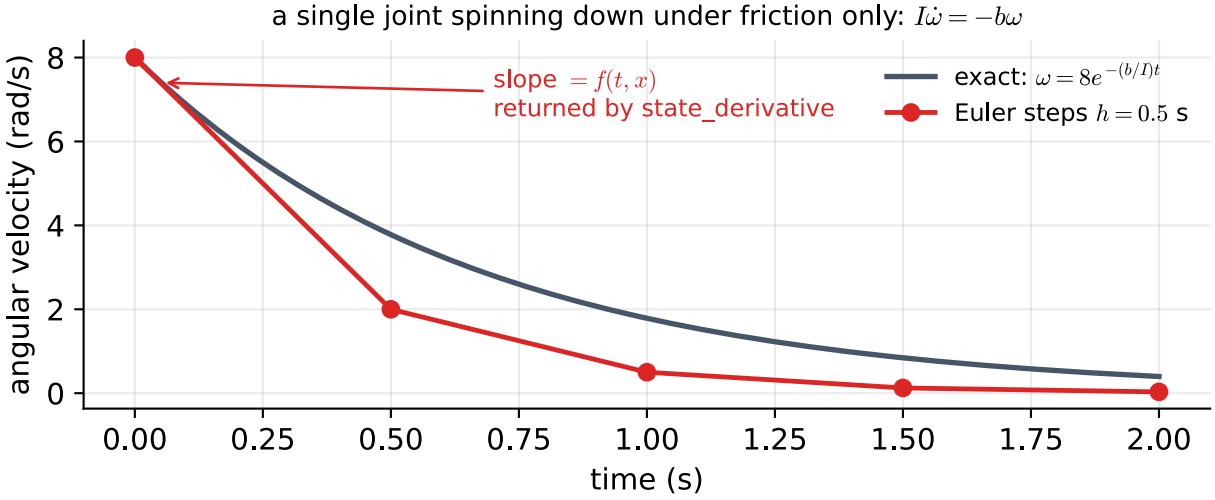
```
def state_derivative(t, x, p, tau_fn):
    q    = x[:2]
    qdot = x[2:]
    tau  = np.asarray(tau_fn(t), dtype=float)
    M    = mass_matrix(q, p)
    rhs  = tau - coriolis_vector(q, qdot, p) - gravity_vector(q, p) - p.b * qdot
    qddot = np.linalg.solve(M, rhs)
    return np.concatenate([qdot, qddot])
```



Where state_derivative sits in the whole project. The dataset generator hands it a torque function, the payload and friction values; the integrator repeatedly calls it to march the state forward in time. The result, a trajectory, is what the DeepONet will later learn to predict directly from the inputs.

2 Toolbox: ODEs and the "state"

An **ordinary differential equation** (ODE) in standard form is $\dot{x} = f(t, x)$: the rate of change of the state vector x is a known function of time and of the current state. If you know $x(0)$ you can in principle follow the arrows forever. The simplest solver, Euler's method, literally does what the quote above says: $x(t + h) \approx x(t) + h\, f(t, x(t))$. RK4 and RK45 (in *solvers.py*) are cleverer versions that call f several times per step to be far more accurate.



An ODE solver in one picture. A joint spinning down under friction alone. At each point, f gives the local slope (red arrow direction); Euler follows it for a step h and re-evaluates. Smaller steps follow the exact grey curve better.

Why the state has four numbers. Newton's law gives *accelerations* (second derivatives): $\ddot{q}_1, \ddot{q}_2$. Solvers want first-order systems. The trick: call the velocities new variables. With $x = [q_1,\ q_2,\ \dot{q}_1,\ \dot{q}_2]$ we have $\dot{x} = [\dot{q}_1,\ \dot{q}_2,\ \ddot{q}_1,\ \ddot{q}_2]$: the first two entries of $\dot{x}$ are simply the last two entries of x, and only the accelerations need to be computed. That is the last line of the function, `np.concatenate([qdot, qddot])`.

3 Assembling the equation of motion

Here we put together the whole Euler-Lagrange system. For each joint k:

$$\underbrace{\frac{d}{dt}\frac{\partial T}{\partial \dot{q}_k} - \frac{\partial T}{\partial q_k}}_{\text{from the mass matrix: } (M\ddot{q})_k + c_k} + \underbrace{\frac{\partial V}{\partial q_k}}_{G_k} = \underbrace{\tau_k - b\,\dot{q}_k}_{Q_k\text{: outside torques}} .$$

The first brace was computed in the *mass_matrix* document ($M\ddot{q}$ terms) and the *coriolis_vector* document (c terms); the second is the *gravity_vector* document. Moving the right-hand pieces around:

$$\boxed{M(q)\,\ddot{q} + c(q,\dot{q}) + G(q) + b\,\dot{q} = \tau} \qquad \Longleftrightarrow \qquad M\,\ddot{q} = \underbrace{\tau - c - G - b\dot{q}}_{\text{“rhs” in the code}}.$$

Where do the two "outside" torques come from?

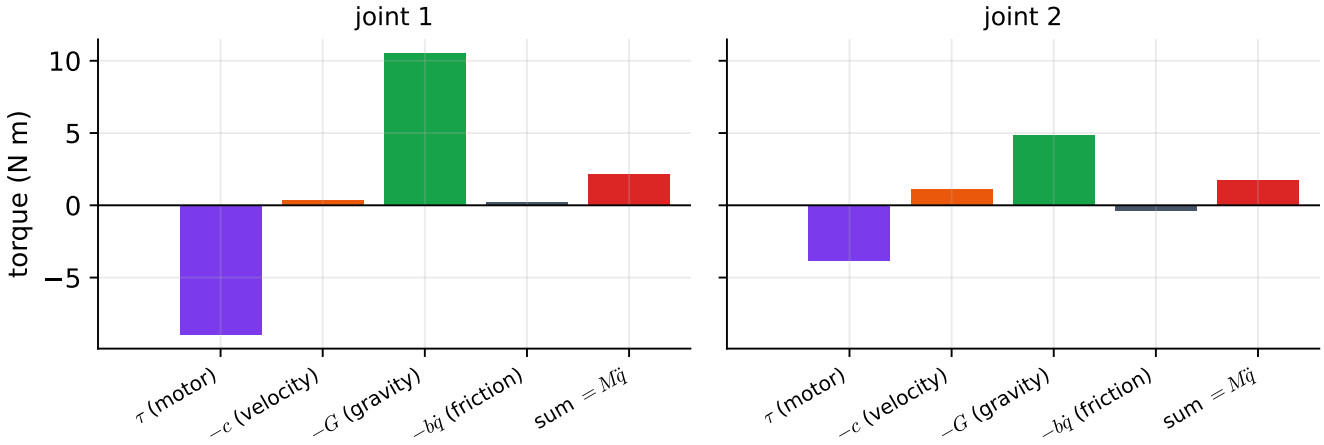
- **Motor torque τ:** a given input. In the dataset it is the sinusoid $\tau_j(t) = A_j \sin(2\pi f_j t + \phi_j)$, produced by `torque_fn` in *data.py*. The function `tau_fn(t)` is how it enters, which is why the argument `t` exists at all: M, c, G do not depend on time directly, but the input does.
- **Viscous friction $-b\dot{q}$:** a drag torque that opposes motion and grows in proportion to the joint speed (like stirring honey), with b in N m s/rad. It is the "generalised force" of friction: the friction torque of joint k acts on q_k with a minus sign because it points against $\dot{q}_k$. The effect alone, with nothing else acting, is $I\dot{\omega} = -b\omega$, hence an exponential spin-down $\omega(t) = \omega_0 e^{-bt/I}$ with time-constant I/b (the grey curve above).

4 Solving for the accelerations

The equation $M\ddot{q} = \mathrm{rhs}$ is a **linear system** in the unknown $\ddot{q}$: two equations, two unknowns. We know M is positive definite (see *mass_matrix*), so it is invertible and the answer exists and is unique: $\ddot{q} = M^{-1}\mathrm{rhs}$. For a 2×2 matrix you can write the answer by hand (Cramer's rule), with $\det M = M_{11}M_{22} - M_{12}^2$:

$$\ddot{q}_1 = \frac{M_{22}\,r_1 - M_{12}\,r_2}{\det M}, \qquad \ddot{q}_2 = \frac{M_{11}\,r_2 - M_{12}\,r_1}{\det M}, \qquad r = \mathrm{rhs}.$$

The code uses `np.linalg.solve(M, rhs)` instead of computing M^{-1} explicitly. Both give the same answer in exact arithmetic, but *solve* is faster and less sensitive to rounding errors (the comment in the code docstring says exactly that). Notice the coupling: even if a torque acts only on joint 2 ($r_1 = 0$), $M_{12} \neq 0$ makes $\ddot{q}_1 \neq 0$: the whole arm responds.



Who is pushing the arm? An actual moment ($t = 1.20$ s) of a dataset-style trajectory. For each joint the bars are the four torques of the equation and their sum $= M\ddot{q}$ (red). Here the motor and gravity dominate; the velocity (Coriolis) and friction terms are smaller but not zero. Which one dominates changes from moment to moment.

5 Worked examples with real numbers

(a) Released from rest, no torque

$m_p = 0.5$ kg, $b = 0.1$, $x = [0, 0, 0, 0]$ (arm held horizontal, at rest), $\tau = 0$. Velocities are zero so $c = 0$ and friction is zero: $\text{rhs} = -G(q) = (-19.6200,\ -4.9050)$ N m. $M = \begin{pmatrix} 1.3267 & 0.4100 \\ 0.4100 & 0.1600 \end{pmatrix}$, $\det M = 0.04417$.

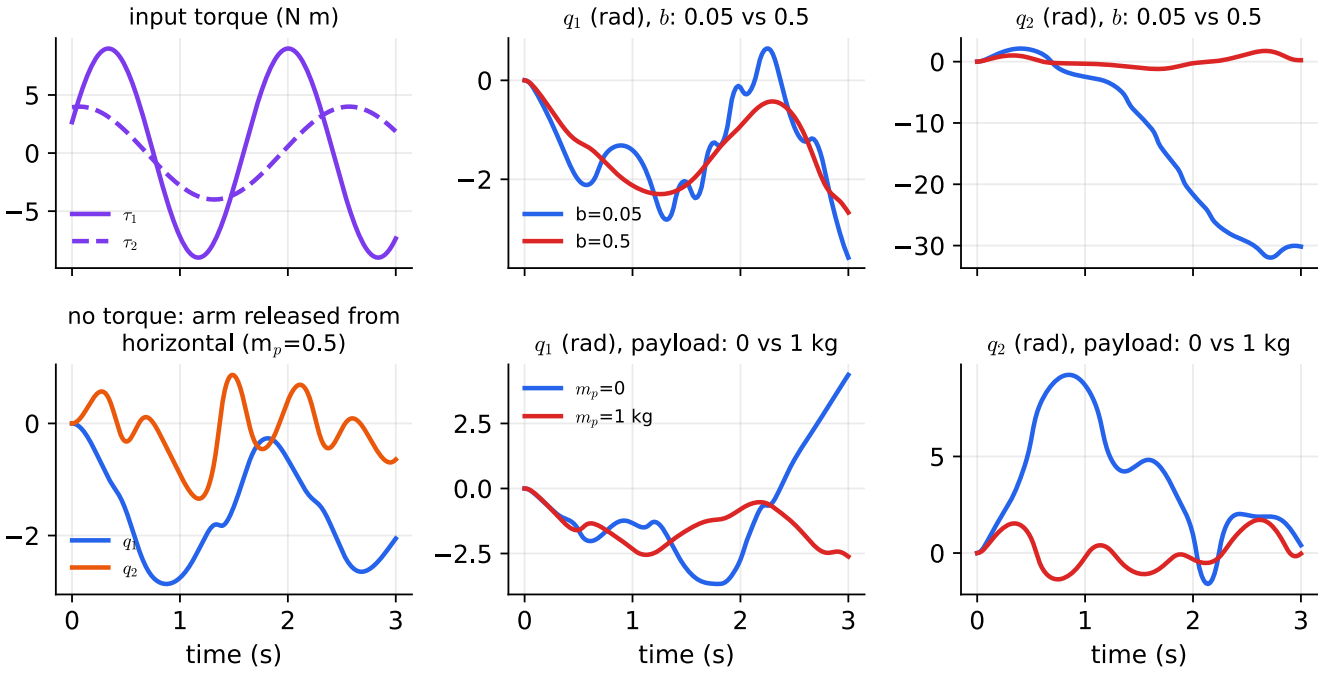
Then

$$\ddot{q}_1 = \frac{0.1600 \cdot (-19.6200) - 0.4100 \cdot (-4.9050)}{0.04417} = -25.543 \text{ rad/s}^2, \qquad \ddot{q}_2 = 34.798 \text{ rad/s}^2.$$

$\ddot{q}_1 < 0$: the whole arm starts to swing down (clockwise), as gravity's lever arms say it must. $\ddot{q}_2 > 0$ is less obvious: q_2 is the angle *relative* to link 1. Link 1 drops quickly, while link 2 (through the coupling M_{12} and its own inertia) lags behind, so the relative angle *opens*. (In absolute terms link 2 has angle $q_1 + q_2$ and acceleration $\ddot{q}_1 + \ddot{q}_2 = 9.25$ rad/s^2, also positive: link 1 yanks the elbow downwards faster than link 2 alone would fall.) The simulated drop is plotted below.

(b) A moving state with torque

$x = [0.5,\ 0.8,\ 2.0,\ -1.0]$, $\tau = (3,\ 1)$ N m. The code evaluates $M = \begin{pmatrix} 1.1750 & 0.3342 \\ 0.3342 & 0.1600 \end{pmatrix}$ and $\text{rhs} = (-11.9637,\ -0.9294)$, solves to $\ddot{q} = (-21.009,\ 38.071)$ and returns $\dot{x} = [2.0,\ -1.0,\ -21.009,\ 38.071]$, the first two numbers are copied velocities, the last two are the solved accelerations.



Simulated trajectories from the real code (RK4 with these `state_derivative` calls inside). Top-left: the input torques. Right columns: effect of friction (top) and of payload (bottom) on the joint angles for the same torque. Bottom-left: a release from horizontal with no torque, a damped double pendulum. More friction or a heavier payload clearly changes the motion; this dependence on (m_p, b) and on the torque *function* is what the DeepONet must learn.

6 Code ↔ maths dictionary

code	maths
`q = x[:2]; qdot = x[2:]`	unpack $x = [q, \dot{q}]$
`tau = tau_fn(t)`	input torque $\tau(t)$
`M = mass_matrix(q, p)`	$M(q)$
`rhs = tau - coriolis - gravity - p.b*qdot`	$\tau - c - G - b\dot{q}$
`np.linalg.solve(M, rhs)`	$\ddot{q} = M^{-1}(\tau - c - G - b\dot{q})$
`np.concatenate([qdot, qddot])`	$\dot{x} = [\dot{q},\ \ddot{q}]$

7 Things worth knowing

- **Units.** Every term in the rhs is in N m; M is in kg m^2; so $\ddot{q}$ comes out in rad/s^2.
- **Equilibrium test.** At $q = (-\pi/2, 0)$, $\dot{q} = 0$, $\tau = 0$: $G = 0$, so $\dot{x} = 0$ (the arm hangs still); the project's tests check this.
- **Time-dependence.** Only through $\tau(t)$. If τ is constant the system is "autonomous" and `t` would be ignored.
- **Sign of friction.** Because friction is subtracted on the right, a joint with positive $\dot{q}_k$ gets a negative contribution: it decelerates. Energy is lost, see the *energy* document.

CHECK YOURSELF

1. Why can't we just integrate $\ddot{q}$ directly with RK4? *Answer:* RK4 works on first-order systems $\dot{x} = f(t, x)$; we turn the second-order law into one by adding the velocities to the state.
2. If M were diagonal ($M_{12} = 0$), what would Cramer's rule reduce to? *Answer:* $\ddot{q}_k = r_k / M_{kk}$: independent joints, $\tau = I\ddot{\theta}$ for each.
3. With $\tau = 0$ and no gravity, what does friction do to a spinning arm? *Answer:* it decays the velocities exponentially; $M\ddot{q} = -b\dot{q}$.